



Fullstack Web Development

Session 7 | Week 6/Day 2 | 120 min

React JS - Part 01

Instructor: Ms. Liev Nova



090667393



ReactJS

What is **React**?

React is a **JavaScript library** used to build user interfaces (UI).

It is:

- Open-source
- Created by Meta (Facebook)
- Widely used for modern web applications

React = Build UI using reusable components

Why use React?

React helps :

- build fast applications
- reuse code (components)
- manage UI easily
- create dynamic web apps

ReactJS

Component-Based Architecture

React uses **components**

Component = small reusable UI block

Example:

- Navbar
- Button
- Card
- Footer

Example Structure:

App

|—— Navbar

|—— Hero

|—— Card

|—— Footer

ReactJS

What is Virtual DOM?

React uses a **Virtual DOM**

Virtual DOM = copy of real DOM

How it works:

1. React creates Virtual DOM
2. Detects changes
3. Updates only changed parts

Benefit:

- Faster performance
- Efficient updates

Example: (Concept)

```
const element = <h1>Hello</h1>;
```

React updates only this part instead of reloading whole page

ReactJS

One-Way Data Flow

React uses **one-way data flow**

Data flows from: Parent → Child

Why?

- easier to debug
- predictable behavior
- better control

Example: (Concept)

```
function Parent() {  
  return <Child name="Dara" />;  
}  
  
function Child(props) {  
  return <h1>{props.name}</h1>;  
}
```


ReactJS

Key Features of React

- Component-based
- Virtual DOM
- One-way data binding
- Reusable code
- Fast performance

React is used by:

- Facebook
- Instagram
- Netflix

React is:

- a JavaScript library
- used to build UI
- fast because of Virtual DOM
- uses components
- uses one-way data flow

Setting Up React Environment

Setting Up React (Using Vite)

Before using React, we need to prepare the development environment.

Install Node.js

React requires Node.js and npm

Check Installation

Open terminal and run:

```
node -v  
npm -v
```

If **Node.js** is installed, you will get the result :

```
C:\Users\lievnova>node -v  
v22.19.0
```

```
C:\Users\lievnova>npm -v  
10.9.3
```

Setting Up React Environment

Setting Up React (Using Vite)

What is npm?

- npm = Node Package Manager
- used to install libraries
- comes with Node.js

Install a Build Tool (Vite) : `npm install -g create-vite`

Global Installation



```
E:\ReactJS>npm install -g create-vite  
added 1 package in 1s  
  
1 package is looking for funding  
run 'npm fund' for details
```


React Installation

Create a React Project with template React:

```
npm create vite@latest my-react-app -- --template react
```

- If you get the message, just press *Enter*.
- And you will get messages :

```
Need to install the following packages:  
create-vite@6.5.0  
Ok to proceed? (y)
```

```
E:\ReactJS>npm create vite@latest my-react-app -- --template react  
  
> npx  
> create-vite my-react-app --template react  
  
|  
* Use rolldown-vite (Experimental)?:  
  Yes  
  > No  
|
```

Press Enter

```
E:\ReactJS>npm create vite@latest my-react-app -- --template react  
  
> npx  
> create-vite my-react-app --template react  
  
|  
o Use rolldown-vite (Experimental)?:  
  No  
|  
* Install with npm and start now?  
  Yes / > No  
|
```

Press Enter

React Installation

Create a React Project:

```
npm create vite@latest my-react-app -- --template react
```

- And if creation was success, you will get this result .

```
E:\ReactJS>npm create vite@latest my-react-app -- --template react
> npx
> create-vite my-react-app --template react

|
|  Use rolldown-vite (Experimental)?:
|  No
|
|  Install with npm and start now?
|  No
|
|  Scaffold project in E:\ReactJS\my-react-app...
|
- Done. Now run:

cd my-react-app
npm install
npm run dev
```

**Note :*

```
npm install
```

Install all dependencies.

```
npm run dev
```

Run the project.

React Installation

Create a React Project using vite: `npm create vite@latest`

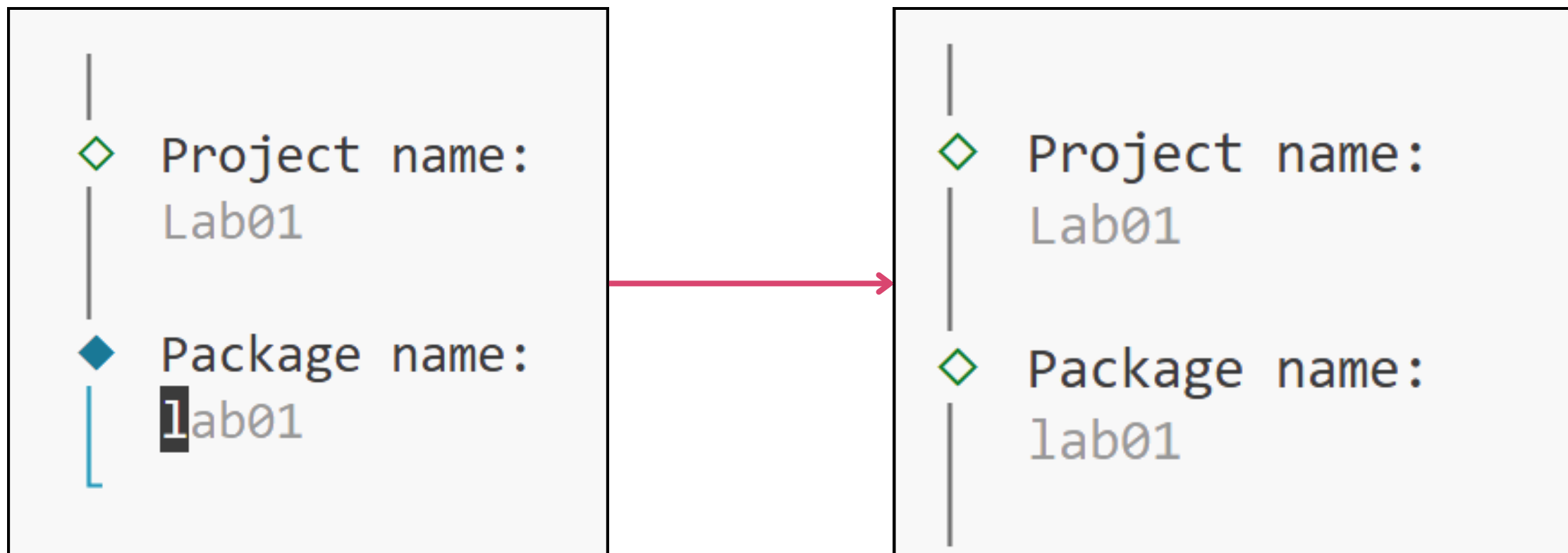
- Create Project Step-by-Step
 - **Step01:** run *npm create vite@latest*

```
PS E:\Teach\Week 03\Day02\Week05_Day01> npm create vite@latest  
  
> npx  
> create-vite  
  
|  
◆ Project name:  
| vite-project
```

React Installation

Create a React Project using vite: `npm create vite@latest`

- Create Project Step-by-Step
 - **Step02:** Put the Project Name and Package Name

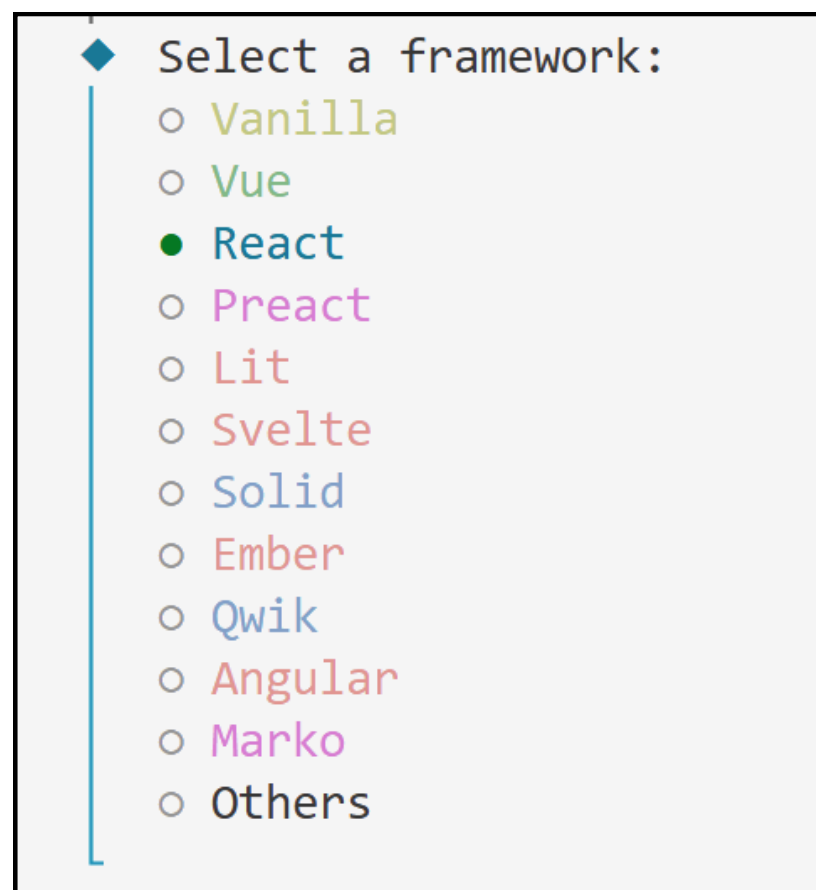


React Installation

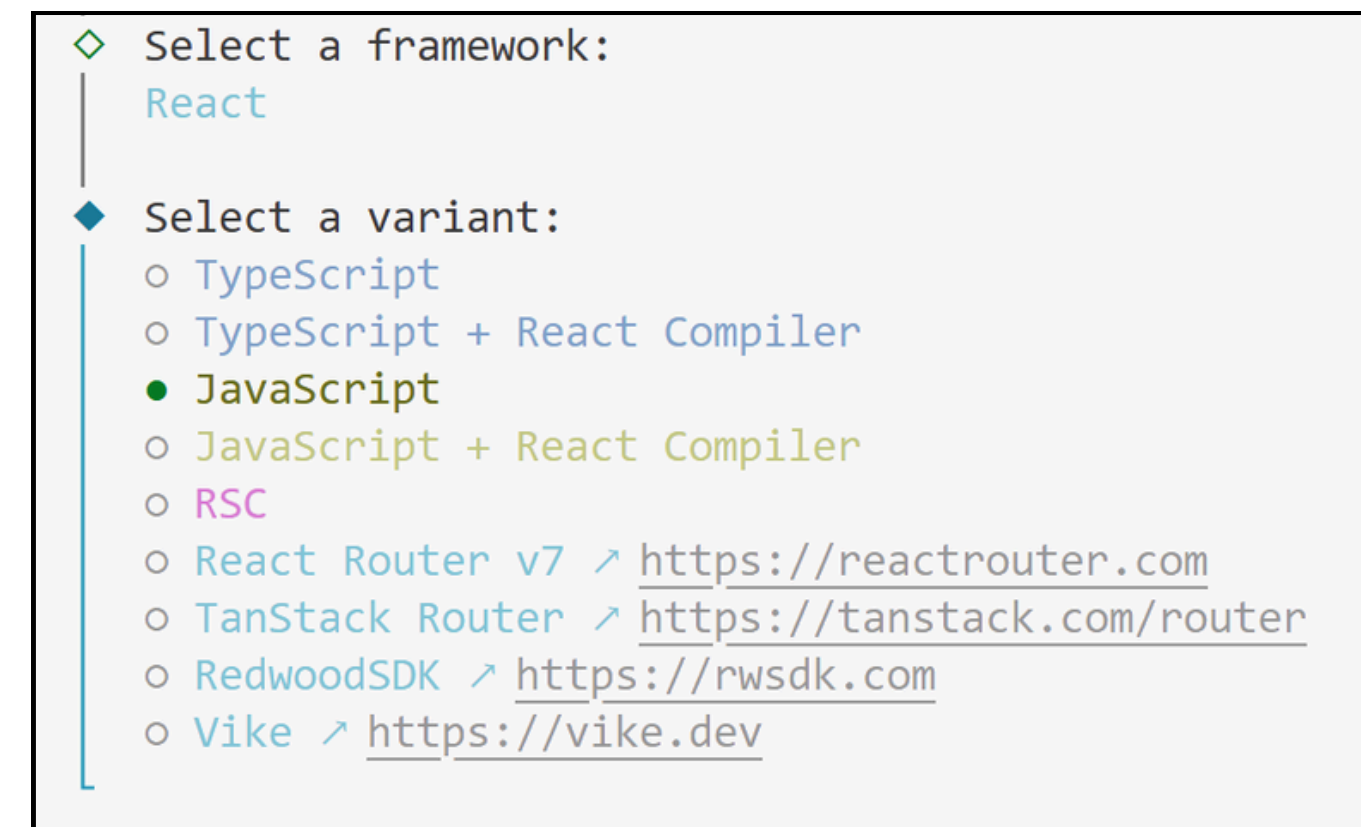
Create a React Project using vite:

```
npm create vite@latest
```

- Create Project Step-by-Step
 - **Step03:** Select a framework



- **Step04:** Select a variant



React Installation

Create a React Project using vite:

```
npm create vite@latest
```

- Create Project Step-by-Step
 - **Step05:** Confirm Installation

```
◆ Install with npm and start now?  
|  
| ● Yes / ○ No
```

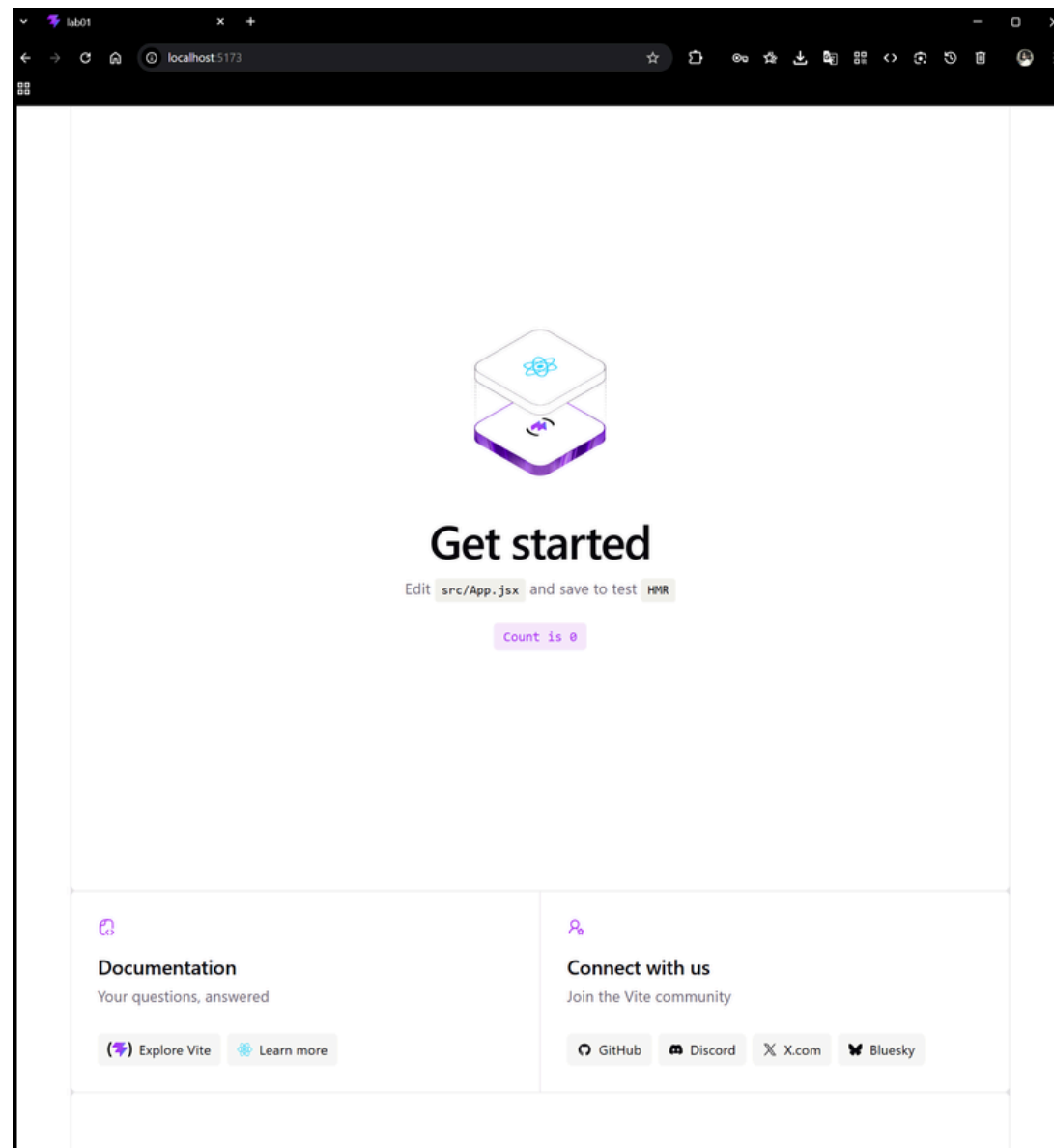


```
◇ Install with npm and start now?  
| Yes  
◇ Scaffolding project in E:\Teach\Week 03\Day02\Week05_Day01\Lab01...  
|  
◇ Installing dependencies with npm...  
  
added 151 packages, and audited 152 packages in 29s  
  
36 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities  
|  
◇ Starting dev server...  
  
> lab01@0.0.0 dev  
> vite  
  
VITE v8.0.3 ready in 2977 ms  
  
→ Local:   http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```

React Installation

View your project running result on browser : <http://localhost:5173/>

- The Result



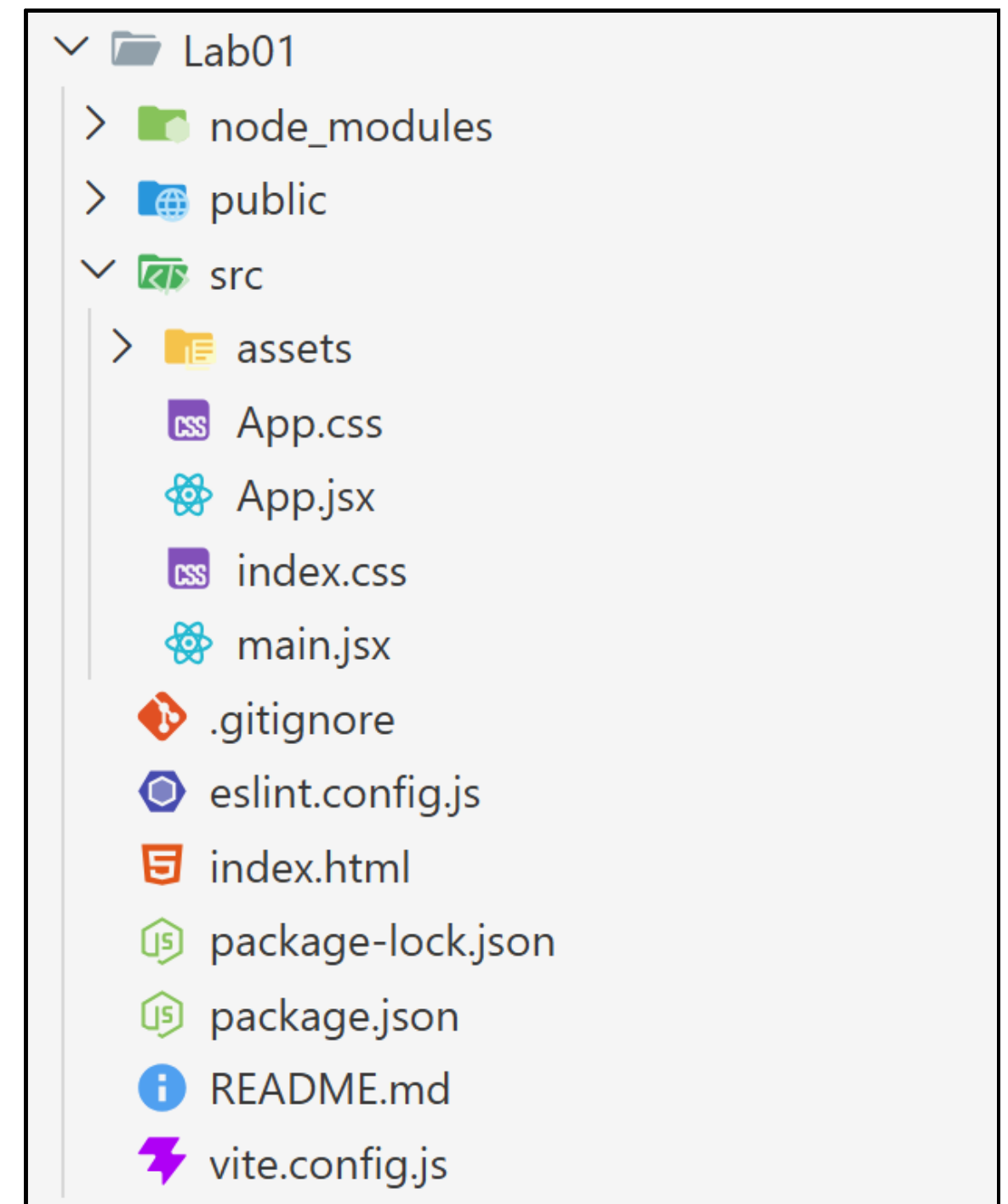
Project Structure

React Project Structure (Vite)

When we create a React app, it generates a folder structure.

Important Folders:

- *src/* — Main working folder
 - contains components
 - contains styles
 - where most coding happens
- *public/* — Static files
 - images
 - icons
 - favicon



Project Structure

React Project Structure (Vite)

When we create a React app, it generates a folder structure.

Important Files:

- *main.jsx* — (Entry Point) : React starts here
 - loads React
 - renders *<App />* into HTML

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

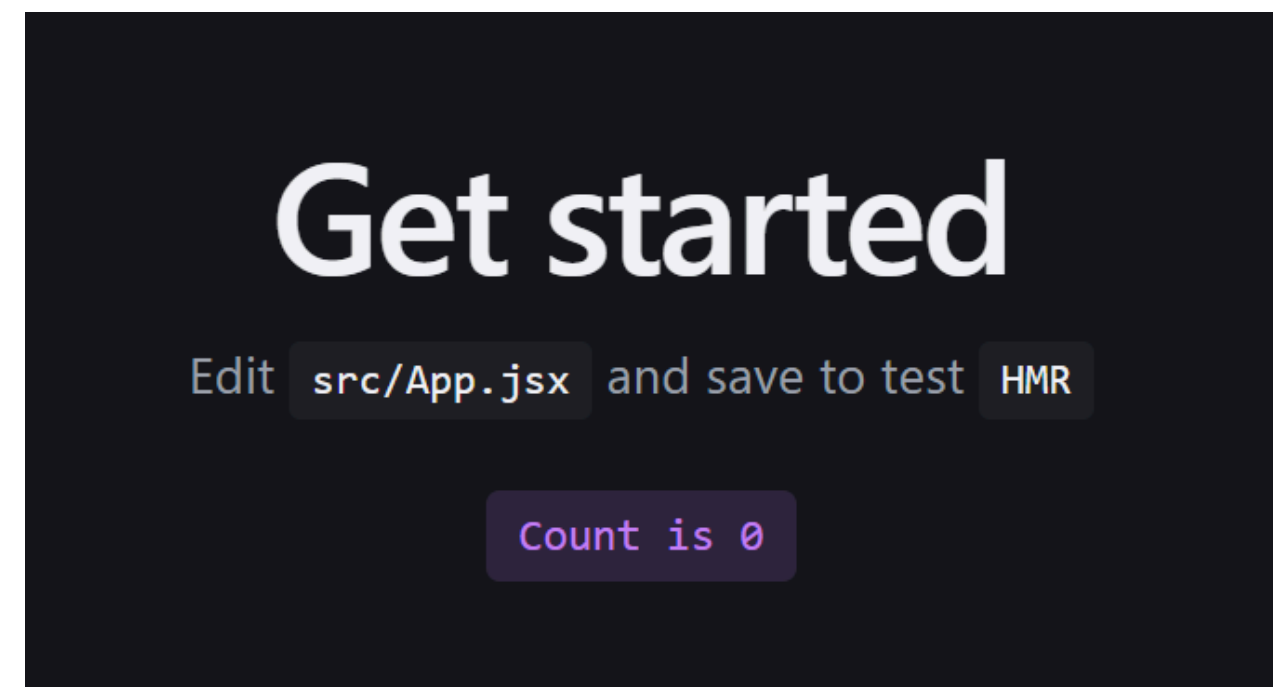
Project Structure

React Project Structure (Vite)

When we create a React app, it generates a folder structure.

Important Files:

- *App.jsx* — (Main Component) : This is the main UI component
 - controls what appears on screen
 - root component
- *App.css* — (Styles) : Used for styling your app



Project Structure

React Project Structure (Vite)

When we create a React app, it generates a folder structure.

Important Files:

- *package.json* : Project configuration file
 - Contains:
 - dependencies
 - scripts
 - project info

```
{
  "name": "lab01",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.4",
    "react-dom": "^19.2.4"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.4",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^9.39.4",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.4.0",
    "vite": "^8.0.1"
  }
}
```

Project Structure

React Project Structure (Vite)

When we create a React app, it generates a folder structure.

Important Files:

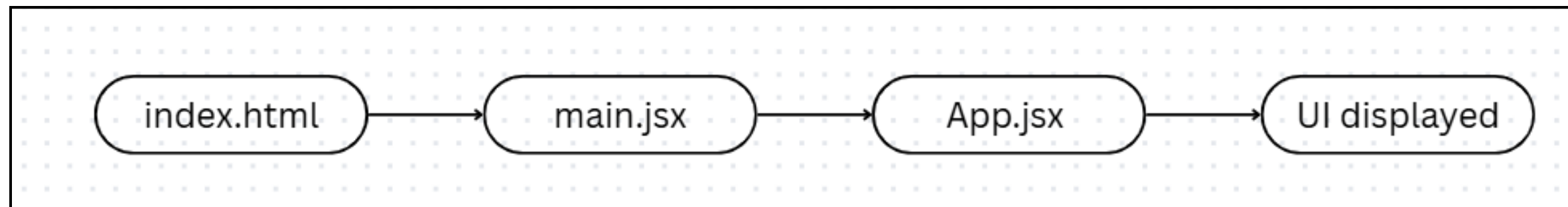
- *vite.config.js* : Configuration for Vite
 - Used for:
 - plugins
 - build settings

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

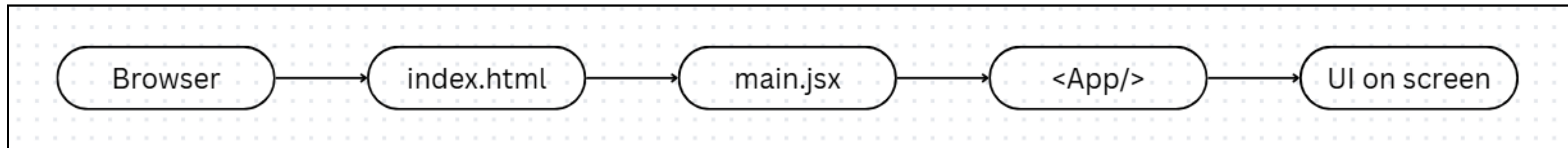
// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

React Flow

How React Works (Flow)



Visual Flow



Key Concept

- React app starts from:

`<App />`

What is JSX ?

JSX (JavaScript XML)

JSX is a syntax that allows us to write **HTML inside JavaScript**

Simple Definition — JSX = JavaScript + HTML-like syntax

Why JSX?

- JSX makes React code:
 - easier to read
 - easier to write
 - similar to HTML

JSX Example:

```
const greeting = <h1>Hello</h1>;
```

- This looks like HTML
- But it is actually JavaScript

What is JSX ?

Important Note

- JSX is NOT real HTML
- React converts JSX into JavaScript

Behind the Scene

```
const greeting = <h1>Hello</h1>;
```

```
React.createElement("h1", null, "Hello");
```



React converts to:

What is JSX ?

JSX Rule

1. Single Root Element

```
return (  
  <h1>Title</h1>  
  <p>Content</p>  
)
```

✗ WRONG – two top-level elements

```
return (  
  <div>  
    <h1>Title</h1>  
    <p>Content</p>  
  </div>  
)
```

✓ CORRECT

```
return (  
  <>  
    <h1>Title</h1>  
    <p>Content</p>  
  </>  
)
```

Alternative (Fragment)

*No extra **<div>** needed*

What is JSX ?

JSX Rule

2. className

```
<div class="card">Content</div>
```

✗ WRONG

```
<div className="card">Content</div>
```

Because `class` is reserved in JavaScript

3. Close Tags

```

```

✗ WRONG

```

```

✓ *CORRECT*

All tags must be closed

4. CamelCase — Use camelCase for attributes

```
<button onclick="clickMe()"></button>
```

✗ WRONG

```
<button onClick={clickMe}></button>
```

✓ *CORRECT*

What is JSX ?

JSX Rule

5. JavaScript Expressions — Use `{ }` to write JavaScript inside JSX

- Example:

```
const name = "Student";  
const message = <h1>Hello, {name}!</h1>;
```

Hello, Student!

```
return (  
  <>  
    <h1>{message}</h1>  
  </>  
)
```

What is JSX ?

JSX Rule

5. JavaScript Expressions — Use `{ }` to write JavaScript inside JSX

- Expression Examples

```
const number = 10;

return (
  <>
    <p>10 + 5 = {10 + 5}</p>
    <p>Double: {number * 2}</p>
    <p>Result: {number > 35 ? "Big" : "Small"}</p>
  </>
);
```

10 + 5 = 15
Double: 20
Result: Small



What is a Component?

React Component

A component is: A reusable piece of UI (like a template)

Component = small part of a website

Examples:

- Navbar
- Button
- Card
- Page

Why Components?

- reusable
- organized code
- easier to maintain

Component Example:

```
function Hello() {  
  return <h1>Hello React</h1>;  
}
```

What is a Component?

Real Component (HomePage) — *src/pages/HomePage.jsx*

```
import profilePic from '../assets/profile.jpg';
import stemBuilding from '../assets/stem.jpeg';

export default function HomePage() {
  return (
    <main className="flex-1 flex flex-col items-center justify-center space-y-6">

      <section className="flex text-start px-4 space-x-4">

        <img
          src={profilePic}
          alt="Profile"
          className="w-40 h-40 rounded-full shadow-lg border-4 border-white mb-6"
        />

        <div className="space-y-4">
          <h1 className="text-4xl md:text-5xl font-bold">
            Welcome to My React Site
          </h1>

          <p className="text-lg md:text-xl">
            Hello! I'm learning <span className="font-semibold text-blue-600">React</span>.
          </p>
        </div>

      </section>

      <img
        src={stemBuilding}
        alt="Stem Building"
        className="max-w-3xl"
      />

    </main>
  );
}
```


What is a Component?

Using Component in App.jsx

```
import HomePage from './pages/HomePage.jsx';

function App() {
  return (
    <>
      <HomePage />
    </>
  );
}

export default App;
```

What is a Component?

Component Rule

Component name MUST start with capital letter

```
function HomePage() {}
```

```
function homepage() {}
```

x WRONG

What is a Component?

Export Types:

1. Named Export
2. Default Export

What is a Component?

Export Types:

1. Named Export

- Example:

```
export function HomePage() {}
```



```
import { HomePage } from "../pages/HomePage.jsx";
```

What is a Component?

Export Types:

1. Named Export

- Many Named Export Example:

```
export function Card() {}  
export function Button() {}  
export function Title() {}
```



```
import { Card, Button, Title } from "../components/ui/components";
```

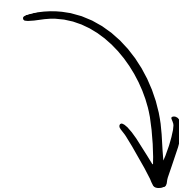
What is a Component?

Export Types:

2. Default Export

- Example:

```
export default function HomePage() {}
```



```
import HomePage from "../pages/HomePage.jsx";
```

What is a Component?

Export Types:

- Difference between **Named Export VS. Default Export**

Feature	Named Export	Default Export
Import syntax	{ } required	no { }
Number per file	many	only one
Name	must match	can rename

- **Example Rename**

```
import MyPage from "./HomePage.jsx";
```

Works because it is default export

What is a Component?

Building with Multiple Components Example:

```
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import HomePage from './pages/HomePage';

function App() {
  return (
    <div>
      <Navbar />
      <HomePage />
      <Footer />
    </div>
  );
}

export default App;
```

Best Practice:

- *split UI into components*
- *keep components small*
- *reuse components*

What is a Component?

Building with Multiple Components Example:

```
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import HomePage from './pages/HomePage';

function App() {
  return (
    <div>
      <Navbar />
      <HomePage />
      <Footer />
    </div>
  );
}

export default App;
```

Component Mistakes:

- Forgetting to export a component

// Wrong

```
function HomePage() {
}
```

// Correct

```
Qodo: Test this function
export function HomePage() {
}
```

- Component name doesn't start with capital letter.

// Wrong

```
function homePage() {
}
```

// Correct

```
function HomePage() {
}
```

What is Routing?

React Routing

Routing means: Navigation between different pages (components) in your app

/ → Home Page

/login → Login Page

/register → Register Page

Each URL shows a different component

Why Routing?

- No page reload
- Faster navigation
- Better user experience

Setting up React Router

Install React Route:

```
npm install react-router-dom
```

Setup Router (main.jsx)

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <BrowserRouter>  
      <App />  
    </BrowserRouter>  
  </React.StrictMode>  
);
```

- For enables routing feature throughout your app.

Setting up React Router

Install React Route:

```
npm install react-router-dom
```

Define Routes (App.jsx)

Route Explanation

Path	Component
/	HomePage
/register	RegisterPage
/login	LoginPage

```
import { Routes, Route } from "react-router-dom";

import HomePage from "../pages/HomePage.jsx";
import LoginPage from "../pages/LoginPage.jsx";
import RegisterPage from "../pages/RegisterPage.jsx";

function App() {
  return (
    <div className="flex flex-col min-h-screen">

      <Navbar />

      <main className="flex-1 flex flex-col">
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/register" element={<RegisterPage />} />
          <Route path="/login" element={<LoginPage />} />
        </Routes>
      </main>

      <Footer />

    </div>
  );
}

export default App;
```

Setting up React Router

Navigation with Link — Use `<Link>` instead of `<a>`

Example:

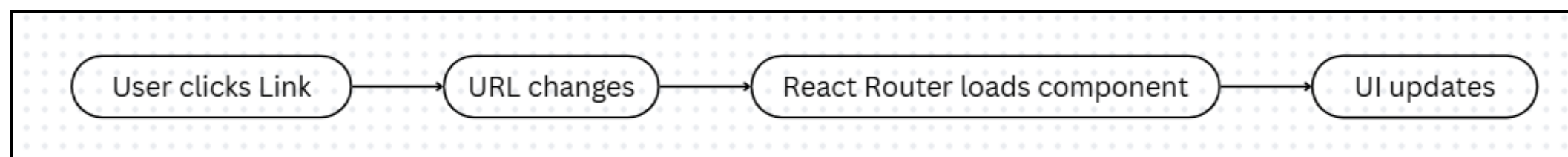
```
import {Link} from "react-router-dom";
Qodo: Test this function
export default function Header() {
  return (
    <header>
      <h1>MyReactApp</h1>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/register">Register</Link>
        <Link to="/login">Login</Link>
      </nav>
    </header>
  );
}
```

Why **NOT** use `<a>`?

- `<a>` reloads the page
- `<Link>` does NOT reload

Setting up React Router

Routing Flow:



Common Mistakes

- Forget to wrap `<BrowserRouter>`
- Forget to import `Routes` and `Route`
- Use `<a>` instead of `<Link>`
- Wrong path



THANK YOU

For your attention !

